

PRACTICAL_WEEK-08

Task 1: Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
UW PICO 5.09

calloc_ptr = (int *)calloc(3, sizeof(int));
if (calloc_ptr == NULL)
{
    printf("calloc failed\n");
    free(malloc_ptr);
    return 1;
}

printf("\nMemory allocated using calloc()\n");

for (int i = 0; i < 3; i++)
{
    printf("calloc_ptr[%d] = %d\n", i, calloc_ptr[i]);
}

/* realloc */
malloc_ptr = (int *)realloc(malloc_ptr, 5 * sizeof(int));

if (malloc_ptr == NULL)
{
    printf("realloc failed\n");
    free(calloc_ptr);
    return 1;
}

printf("\nMemory resized using realloc()\n");

malloc_ptr[3] = 40;
malloc_ptr[4] = 50;

for (int i = 0; i < 5; i++)
{
    printf("malloc_ptr[%d] = %d\n", i, malloc_ptr[i]);
}

/* free */
free(malloc_ptr);
free(calloc_ptr);

printf("\nMemory successfully released using free()\n");

return 0;
```

Fig 1: C program demonstrating dynamic memory allocation using malloc(), calloc(), realloc(), and free()

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/week8_task1.c -o bin/week8_task1
amrutha@Amruthas-MacBook-Air OSSP % ./bin/week8_task1
Dynamic Memory Allocation Demo

Memory allocated using malloc()
malloc_ptr[0] = 10
malloc_ptr[1] = 20
malloc_ptr[2] = 30

Memory allocated using calloc()
calloc_ptr[0] = 0
calloc_ptr[1] = 0
calloc_ptr[2] = 0

Memory resized using realloc()
malloc_ptr[0] = 10
malloc_ptr[1] = 20
malloc_ptr[2] = 30
malloc_ptr[3] = 40
malloc_ptr[4] = 50

Memory successfully released using free()
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 2: Execution output showing dynamic memory allocation, resizing, and deallocation

```

UW PICO 5.09

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr;

    ptr = (int *)malloc(5 * sizeof(int));

    if (ptr == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
    {
        ptr[i] = (i + 1) * 10;
        printf("ptr[%d] = %d\n", i, ptr[i]);
    }

    printf("\nMemory allocated but not freed.\n");
    printf("This creates an intentional memory leak.\n");

    /* free(ptr); intentionally omitted */

    return 0;
}

```

Fig 3: C program containing an intentional memory leak for memory-leak analysis

```

amrutha@Amruthas-MacBook-Air OSSP % gcc src/week8_task1_leak.c -o bin/week8_task1_leak
./bin/week8_task1_leak
ptr[0] = 10
ptr[1] = 20
ptr[2] = 30
ptr[3] = 40
ptr[4] = 50

Memory allocated but not freed.
This creates an intentional memory leak.
amrutha@Amruthas-MacBook-Air OSSP %

```

Fig 4: Execution of the program containing an intentional memory leak

```

amrutha@Amruthas-MacBook-Air OSSP % leaks --atExit -- ./bin/week8_task1_leak
week8_task1_leak(26408) MallocStackLogging: could not tag MSL-related memory as no_footprint, so those pages will be included in process footprint - No such file or directory (2)
week8_task1_leak(26408) MallocStackLogging: recording malloc (and VM allocation) stacks using lite mode
ptr[0] = 10
ptr[1] = 20
ptr[2] = 30
ptr[3] = 40
ptr[4] = 50

Memory allocated but not freed.
This creates an intentional memory leak.
Process 26408 is not debuggable. Due to security restrictions, leaks can only show or save contents of readonly memory of restricted processes.

Process:          week8_task1_leak [26408]
Path:             /Users/USER/*/week8_task1_leak
Load Address:     0x182294000
Identifier:       week8_task1_leak
Version:          0
Code type:        ARM64
Platform:         macOS
Parent Process:   leaks [26407]
Target Type:      live task

Date/Time:        2026-09-20 21:35:25.114 +0530
Launch Time:      2026-09-20 21:35:25.087 +0530
OS Version:       macOS 26.6.2 (25G83)
Report Version:   7
Analysis Tool:    /usr/bin/leaks

Physical footprint: 2000K
Physical footprint (peak): 2000K
Idle exit:        untracked

-----
leaks Report Version: 4.0, multi-line stacks
Process 26408: 191 nodes malloced for 16 KB
Process 26408: 1 leak for 32 total leaked bytes.

STACK OF 1 INSTANCE OF 'ROOT LEAK: <malloc in main>':
2  dyld                                0x1834d04e4 start + 6992
1  week8_task1_leak                    0x182294478 main + 24
0  libsystem_malloc.dylib              0x1836c71fc _malloc_zone_malloc_instrumented_or_legacy + 152
-----
1 (32 bytes) ROOT LEAK: <malloc in main @xaed028020> [32]

```

Fig 5: Memory leak analysis of the program using the macOS leaks utility



Fig 6: Flowchart for dynamic memory allocation and memory-leak analysis

Observation:

Dynamic memory was successfully allocated using `malloc()` and `calloc()`, resized using `realloc()`, and released using `free()`. An intentional memory leak was also created by omitting `free()` and analyzed using a memory-analysis tool.

Result:

Dynamic memory allocation using `malloc()`, `calloc()`, `realloc()`, and `free()` was implemented successfully. Memory-allocation behavior and memory-leak conditions were studied successfully.

Task 2: Develop a program that demonstrates Copy-on-Write behavior after a `fork()`. Analyze memory usage before and after modifying data in the child process.

```
UW PICO 5.09 File: src/we

    printf("Value before modification: %d\n", value);
    printf("Address in child: %p\n", (void *)&value);

    value = 200;

    printf("Value after modification: %d\n", value);
    printf("Child process completed.\n");
}
else
{
    wait(NULL);

    printf("\n----- Parent Process ----- \n");
    printf("Parent PID: %d\n", getpid());
    printf("Value after child modification: %d\n", value);
    printf("Address in parent: %p\n", (void *)&value);

    printf("Parent value remains unchanged.\n");
}

return 0;
}
```

Fig 7: C program demonstrating Copy-on-Write behavior after `fork()`

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/week8_task2.c -o bin/week8_task2
./bin/week8_task2
Before fork()
Parent PID: 28220
Initial value: 100
Address of value: 0x16dd8afc8

----- Child Process -----
Child PID: 28231
Value before modification: 100
Address in child: 0x16dd8afc8
Value after modification: 200
Child process completed.

----- Parent Process -----
Parent PID: 28220
Value after child modification: 100
Address in parent: 0x16dd8afc8
Parent value remains unchanged.
```

Fig 8: Execution output demonstrating Copy-on-Write between parent and child processes

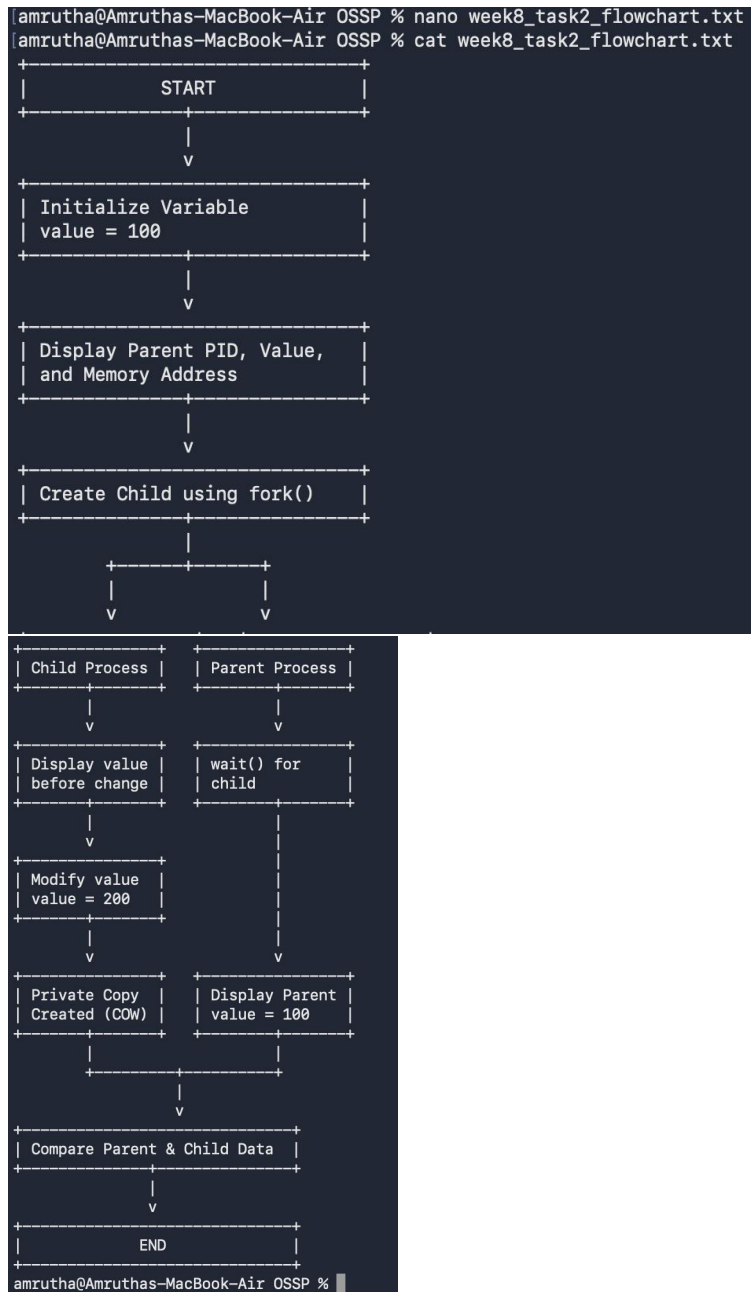


Fig 9&10: Flowchart demonstrating Copy-on-Write behavior between parent and child processes

Observation:

After `fork()`, the parent and child initially had the same value. When the child modified the variable, its value changed to 200 while the parent value remained 100, demonstrating Copy-on-Write behavior.

Result:

Copy-on-Write behavior after `fork()` was demonstrated successfully. The child process received a private copy of the modified memory page, while the parent process retained its original data.